

Complete DSA Learning Order

Topic Content Compilation

Section 10 — Graphs (10.1 – 10.7)

June 30, 2026

Section 10 — Graphs

10.1 — Graph representations — adjacency list vs adjacency matrix

1. PLAIN-LANGUAGE GIST

A graph is a collection of nodes (vertices) connected by edges, and before you can write any graph algorithm, you need to choose how to actually store those connections in memory — the two standard choices, adjacency list and adjacency matrix, trade off memory usage against the speed of certain operations, and picking the right one for a given problem is itself a meaningful design decision.

2. REAL-WORLD ANALOGY

Picture two ways of recording who's friends with whom in a social network. An adjacency list is like each person having their own personal address book, listing only the names of their actual friends — compact, and you only write down real relationships. An adjacency matrix is like a giant spreadsheet with every single person as both a row and a column, where you mark a checkbox at every intersection of two people, friends or not — comprehensive, but mostly full of empty (non-friend) checkboxes once the network gets large.

3. CONNECT TO PRIOR KNOWLEDGE

This is a generalization of 3.1's linked-node concept — a graph node can have any number of connections (not just one Next pointer, or even a fixed two like 8.1's tree nodes), and an adjacency list is essentially “every node has its own dynamic list of neighbors,” directly reusing `List<T>` (0.6) or similar collection types per node. This topic also previews where 1.8's grid traversal and 8.3's tree-level-order traversal generalize: grids and trees are both special, more constrained cases of the general graph structures introduced here.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: An adjacency list stores, for each node, a list of the nodes it's directly connected to — typically a `Dictionary<int, List<int>>` or `List<List<int>>` in C#, mapping each node to its neighbors. An adjacency matrix stores a 2D grid where `matrix[i][j]` is 1 (or a weight, or true) if an edge exists from node *i* to node *j*, and 0/false otherwise — every possible pair of nodes gets an entry, whether or not an edge actually exists between them.

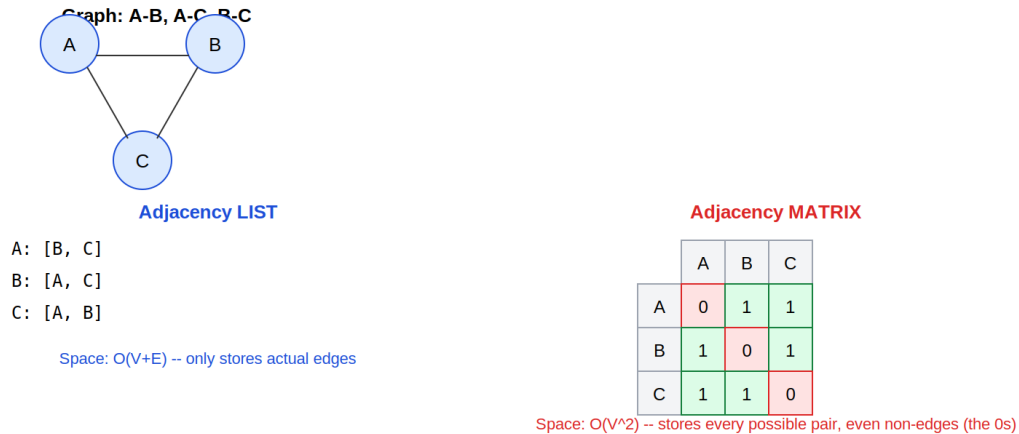
Next layer — the space and time tradeoffs that drive the choice:

- **Space:** an adjacency list uses $O(V + E)$ space (V = vertices, E = edges) — proportional to what's actually there. An adjacency matrix uses $O(V^2)$ space regardless of how many edges actually exist, since it must reserve a slot for every possible pair — for a sparse graph (relatively few edges), this is enormously wasteful; for a dense graph (edges close to the maximum possible), the two representations use comparable space.
- **Checking if a specific edge exists:** an adjacency matrix answers “is there an edge from *A* to *B*?” in $O(1)$ — direct array lookup. An adjacency list must scan through node *A*'s neighbor list, $O(\text{degree of } A)$ in the worst case, which can be $O(V)$ for a highly-connected node.
- **Iterating over all of a node's neighbors** (the single most common graph operation, used by BFS/DFS in 10.3/10.4): an adjacency list does this directly and efficiently, only visiting actual neighbors, $O(\text{degree})$. An adjacency matrix must scan an entire row of size V to find which entries are actually edges, even if that node has very few real neighbors — $O(V)$ regardless of actual degree.

- **The practical rule of thumb:** adjacency lists are the default choice for most graph problems (especially sparse graphs, which most realistic graphs are), while adjacency matrices are preferred specifically when you need frequent $O(1)$ “does this exact edge exist” lookups, or when the graph is genuinely dense enough that the $O(V^2)$ space cost isn't wasteful.

5. VISUAL REPRESENTATION

Same graph, two representations



The diagram shows the identical 3-node graph (A-B, A-C, B-C) represented both ways: the adjacency list stores only the 3 actual edges compactly, while the adjacency matrix reserves all 9 possible cells (including 3 diagonal “self” entries and several non-edges), explicitly showing the $O(V+E)$ vs $O(V^2)$ space difference made concrete.

6. WHEN TO USE / WHEN NOT TO USE

Use an adjacency list when:

- The graph is sparse (E is much smaller than V^2) — true for most realistic graphs (social networks, road networks, dependency graphs rarely have anywhere close to V^2 actual connections).
- You primarily need to iterate over a node's neighbors — the core operation in BFS (10.3), DFS (10.4), and most graph traversal algorithms.

Use an adjacency matrix when:

- You need frequent $O(1)$ “does this specific edge exist” queries, and the graph is small enough or dense enough that $O(V^2)$ space isn't prohibitive.
- You're working with algorithms that are naturally matrix-based (certain specialized algorithms, like some all-pairs shortest path techniques, operate directly on the matrix representation).

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — building both representations for the same small graph, the exact diagram above:

```
// Adjacency list
var adjList = new Dictionary<char, List<char>> {
    ['A'] = new List<char> { 'B', 'C' },
    ['B'] = new List<char> { 'A', 'C' },
    ['C'] = new List<char> { 'A', 'B' }
};

// Adjacency matrix (using indices 0=A, 1=B, 2=C)
int[, ] adjMatrix = {
    { 0, 1, 1 },
    { 1, 0, 1 },
    { 1, 1, 0 }
};
```

INTERMEDIATE — checking if an edge exists, contrasting the cost on each representation:

```
bool HasEdgeList(Dictionary<char, List<char>> adjList, char a, char b) {
    return adjList[a].Contains(b); // O(degree of a) -- 1.1/2.2's tradeoff
}

bool HasEdgeMatrix(int[, ] adjMatrix, int a, int b) {
    return adjMatrix[a, b] == 1; // O(1) -- direct index lookup
}
```

This directly echoes 2.2's `List<T>.Contains()` vs `HashSet<T>.Contains()` tradeoff — `HasEdgeList` could itself be sped up to $O(1)$ average by using a `HashSet<char>` per node instead of a `List<char>`, a worthwhile refinement when edge-existence checks are frequent even while keeping the overall adjacency-list space advantage.

ADVANCED — building an adjacency list from a weighted edge list input, a common real-world parsing step:

```
Dictionary<int, List<(int neighbor, int weight)>> BuildWeightedAdjList(
    int numNodes, List<(int from, int to, int weight)> edges,
    bool directed) {
    var adjList = new Dictionary<int, List<(int, int)>>();
    for (int i = 0; i < numNodes; i++) adjList[i] = new List<(int, int)>();

    foreach (var (from, to, weight) in edges) {
        adjList[from].Add((to, weight));
        if (!directed) {
            adjList[to].Add((from, weight)); // undirected: add BOTH
        }
    }
    return adjList;
}
```

This shows a realistic construction pattern: edges typically arrive as a flat list of (from, to, weight) tuples (e.g., from a file or API input), and building the adjacency list means populating each node's neighbor list — critically, for an undirected graph, each edge must be added in both directions (directly previewing 10.2's directed/undirected distinction), while a directed graph only adds the one specified direction.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, there's a third, less commonly needed representation worth knowing exists: an edge list (just a flat list of (from, to) pairs with no per-node organization at all) — useful primarily as an intermediate input format (as shown in the advanced example) or for algorithms that process edges directly without needing neighbor lookups, like certain sorting-based approaches to minimum spanning trees (Tier 2/3 topics). There's also a practical engineering

note: real-world graph libraries often use a hybrid or more sophisticated representation (compressed sparse row formats, for instance) when performance at scale matters, but adjacency lists and matrices remain the two standard mental models virtually all interview and most practical contexts expect.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is defaulting to an adjacency matrix out of familiarity (it “looks like” the more obvious, table-like representation) without considering the graph's actual density — for the large, sparse graphs common in real interview problems, this wastes significant memory and slows down neighbor iteration, the most frequently needed operation. Another is forgetting to add edges in both directions when building an adjacency list for an undirected graph — adding only `adjList[from].Add(to)` without the corresponding `adjList[to].Add(from)` silently creates a graph that behaves as if it were directed, breaking algorithms that assume undirected traversal works both ways. A third: using a `List<T>` for each node's neighbor list when frequent edge-existence checks are needed, missing the easy $O(1)$ -average upgrade to a `HashSet<T>` per node (2.2's pattern) that preserves the adjacency list's overall space advantage while fixing its slower edge-lookup weakness.

10. SELF-CHECK

- Why does an adjacency matrix always use $O(V^2)$ space regardless of how many edges actually exist, while an adjacency list's space scales with the actual number of edges?
- Why is iterating over a node's neighbors generally faster with an adjacency list than with an adjacency matrix, even though both can answer “what are this node's neighbors”?
- When building an adjacency list for an undirected graph from a list of edges, why must each edge be added in both directions, while a directed graph's edges are added in only one direction?

10.2 — Directed vs undirected, weighted vs unweighted graphs

1. PLAIN-LANGUAGE GIST

Every graph can be classified along two independent axes: whether its edges have a direction (directed: $A \rightarrow B$ doesn't imply $B \rightarrow A$ is also valid; undirected: an edge always works both ways) and whether its edges carry a cost or distance (weighted: each edge has a numeric value; unweighted: edges simply exist or don't, with no associated cost). These two classifications determine which algorithms are even applicable and how the graph must be represented.

2. REAL-WORLD ANALOGY

Picture the difference between a two-way street (undirected — you can drive either direction) and a one-way street (directed — only one direction is legal), and separately, the difference between a free pedestrian path (unweighted — walking it “costs” nothing distinguishable) and a toll road with a specific dollar cost (weighted — each road segment has an explicit price). A city's road network can mix all four combinations: some one-way toll roads, some two-way free streets, and everything in between — the two classifications are entirely independent of each other.

3. CONNECT TO PRIOR KNOWLEDGE

This directly extends 10.1's representation choices — an undirected graph's adjacency list needs edges added in both directions (as flagged in 10.1's common mistake), while a weighted graph's adjacency list needs to store a (neighbor, weight) pair rather than just a bare neighbor, exactly the structure previewed in 10.1's advanced example.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Directed means an edge from A to B doesn't automatically mean there's an edge from B to A — direction matters, and the two are tracked independently. Undirected means every edge is inherently bidirectional — if A connects to B, B connects to A, with no distinction. Weighted means each edge carries an associated numeric value (commonly distance, cost, or capacity). Unweighted means edges are purely binary — present or absent, with no associated value (or equivalently, every edge can be thought of as having an implicit weight of 1).

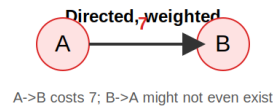
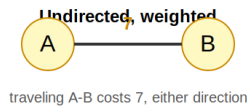
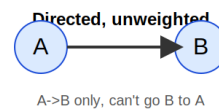
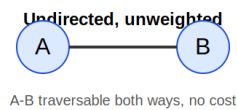
Next layer — why these classifications change which algorithms apply, and how:

- **Directed graphs introduce the possibility of cycles that aren't immediately “undone”** — in an undirected graph, traversing an edge and immediately traversing it back is trivial and meaningless (you're just back where you started); in a directed graph, a genuine cycle ($A \rightarrow B \rightarrow C \rightarrow A$) means something structurally real and detectable, which is exactly why 10.5's cycle detection needs fundamentally different logic for directed versus undirected graphs.
- **Weighted graphs require comparing accumulated costs, not just step counts** — unweighted BFS (10.3) finds the shortest path by counting edges (since every edge “costs” the same), but a weighted graph's “shortest” path must account for varying edge costs, requiring different algorithms entirely (Dijkstra's algorithm, a Tier 2 topic, generalizes BFS's idea using a priority queue, 9.2's structure, instead of a plain queue).
- **A directed acyclic graph (DAG)** — directed, with no cycles — is a specially important combination, since it's the structure that 10.7's topological sort requires: a valid “dependency order” can only exist if there are no circular dependencies, which is precisely what “acyclic” guarantees.
- The same underlying graph can be analyzed differently depending on which classification you apply — a road network might be naturally undirected (most streets) but contain some directed segments (one-way streets), and naturally weighted (distances) but sometimes analyzed as unweighted (just “is there a route,” ignoring distance) depending on the specific

question being asked.

5. VISUAL REPRESENTATION

Four graph flavors: directed/undirected x weighted/unweighted



These two properties are INDEPENDENT -- a graph can be any combination.
Both choices change which algorithms apply and how adjacency is represented.

The diagram shows all four combinations side by side using the same two nodes A and B: undirected-unweighted (a plain bidirectional connection), directed-unweighted (one-way only), undirected-weighted (bidirectional with a cost), and directed-weighted (one-way with a cost) — making concrete that these two classifications combine independently, not as a single linear scale.

6. WHEN TO USE / WHEN NOT TO USE

Recognize directed vs undirected when:

- The problem's relationships are inherently asymmetric (dependencies, one-way streets, "follows" relationships in a social network, prerequisite chains) — that's directed. Symmetric relationships (friendships, physical roads without one-way restrictions, mutual connections) — that's undirected.

Recognize weighted vs unweighted when:

- The problem cares about cost, distance, or capacity, and different edges genuinely differ in that cost — that's weighted, requiring Dijkstra's-style algorithms (Tier 2) rather than plain BFS for shortest-path questions. If the problem only cares about whether a connection exists or the number of steps/hops, treat it as unweighted, where BFS (10.3) directly applies.

Wrong choice / traps to avoid:

- Treating a directed graph's problem as if it were undirected (or vice versa) — this isn't really an optional choice, it's determined by the problem's actual structure; misclassifying it produces an algorithm that solves a different problem than the one actually being asked.
- Applying plain BFS to a weighted graph expecting it to find the shortest path by cost — BFS only guarantees shortest path by edge count, which only coincides with shortest path by cost when all weights are equal (i.e., the graph is effectively unweighted).

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — representing each of the four combinations as adjacency lists:

```
// Undirected, unweighted -- edges added BOTH directions, no weight
var undirectedUnweighted = new Dictionary<int, List<int>>();
void AddUndirectedEdge(int a, int b) {
    undirectedUnweighted[a].Add(b);
    undirectedUnweighted[b].Add(a);
}

// Directed, weighted -- edge added ONE direction, with a weight
var directedWeighted = new Dictionary<int, List<(int to, int weight)>>();
void AddDirectedWeightedEdge(int from, int to, int weight) {
    directedWeighted[from].Add((to, weight));
}
```

INTERMEDIATE — detecting whether a given adjacency list actually represents an undirected graph (a sanity-check utility):

```
bool IsUndirected(Dictionary<int, List<int>> adjList) {
    foreach (var (node, neighbors) in adjList) {
        foreach (int neighbor in neighbors) {
            if (!adjList[neighbor].Contains(node)) {
                return false; // one way but not back -- NOT undirected
            }
        }
    }
    return true;
}
```

This directly checks the defining property from section 4: for every edge $A \rightarrow B$ found, there must also be a corresponding $B \rightarrow A$ — if even one direction is missing, the structure isn't genuinely undirected, regardless of what it was intended to represent.

ADVANCED — converting an unweighted shortest-path problem into BFS, versus recognizing when weights make BFS invalid:

```
// Valid: unweighted graph, BFS correctly finds shortest path by edge count
int ShortestPathUnweighted(Dictionary<int, List<int>> adjList, int start,
                           int target) {
    var queue = new Queue<(int node, int dist)>();
    var visited = new HashSet<int> { start };
    queue.Enqueue((start, 0));

    while (queue.Count > 0) {
        var (node, dist) = queue.Dequeue();
        if (node == target) return dist;
        foreach (int neighbor in adjList[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor);
                queue.Enqueue((neighbor, dist + 1));
            }
        }
    }
    return -1;
}

// INVALID for a WEIGHTED graph: this still counts EDGES, not accumulated
// weight -- it would confidently return a wrong "shortest" path if edges
// have different costs. Dijkstra's algorithm (Tier 2) is the correct
// tool, using a priority queue (9.2) ordered by accumulated cost
// instead of a plain queue ordered by edge count.
```

This explicitly marks the boundary of what BFS can and can't correctly solve — reusing 10.3's BFS-on-unweighted-graphs logic, while flagging that applying the same code to a weighted graph produces a confidently wrong answer rather than an error, which is exactly the kind of

silent failure mode worth recognizing in advance.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, a directed graph can be converted into something resembling an undirected graph by adding both directions for every edge (effectively doubling the edge count), but this is rarely meaningful — the resulting structure loses real information about directionality that often matters for the problem's actual semantics, so this “trick” is more of a curiosity than a genuinely useful transformation. There's also a fundamental graph-theory distinction worth knowing by name: a directed graph's “degree” splits into in-degree (edges pointing in) and out-degree (edges pointing out) — a distinction with no counterpart in undirected graphs, and one that becomes directly relevant in 10.7's topological sort, which specifically tracks in-degree to determine valid processing order.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming all graphs are undirected by default and forgetting to add edges in both directions when the problem actually describes a directed relationship — silently treating a one-way dependency as bidirectional produces a structurally different (and incorrect) graph. Another is applying unweighted BFS to a weighted shortest-path problem, expecting it to “still work approximately” — it doesn't approximate the correct answer, it can be confidently, specifically wrong whenever edge weights differ meaningfully, since BFS's correctness depends entirely on every edge being equivalent in cost. A third: not recognizing that “directed and acyclic” (a DAG) is a specific, named, important combination — treating a general directed graph the same as a DAG misses that topological sort (10.7) and several other algorithms specifically require and exploit the acyclic property, which isn't guaranteed just because a graph is directed.

10. SELF-CHECK

- Why are “directed vs. undirected” and “weighted vs. unweighted” independent classifications, rather than two ends of a single spectrum?
- Why does plain BFS correctly find the shortest path in an unweighted graph, but potentially give a wrong answer if naively applied to a weighted graph with varying edge costs?
- Why is “directed and acyclic” (a DAG) specifically the structural requirement for topological sort (10.7) to be possible at all, rather than just “directed” alone being sufficient?

10.3 — Breadth-First Search (BFS) — mechanics and use cases

1. PLAIN-LANGUAGE GIST

BFS explores a graph level by level outward from a starting node — visiting everything at distance 1 before anything at distance 2, and so on — using a queue to enforce this strict ordering, which is exactly 8.3's level-order tree traversal generalized from trees to arbitrary graphs, now requiring an explicit “visited” check since general graphs (unlike trees) can have cycles.

2. REAL-WORLD ANALOGY

Picture ripples spreading outward from a stone dropped in a pond — every point at a given distance from the impact is reached at the same “wave,” before the next, slightly farther wave arrives. BFS explores a graph in exactly this expanding-wave pattern, guaranteeing that anything closer to the start is always discovered before anything farther away.

3. CONNECT TO PRIOR KNOWLEDGE

This is the direct, full generalization of 8.3's tree-specific level-order traversal and 4.2's original BFS preview (on a grid) — the queue-based mechanism is identical, but general graphs introduce a genuinely new requirement: an explicit visited set (2.5's pattern), since unlike a tree's guaranteed-acyclic structure, a general graph can have cycles that would cause infinite re-exploration without that tracking.

4. CORE MECHANISM (Simple → Intermediate)

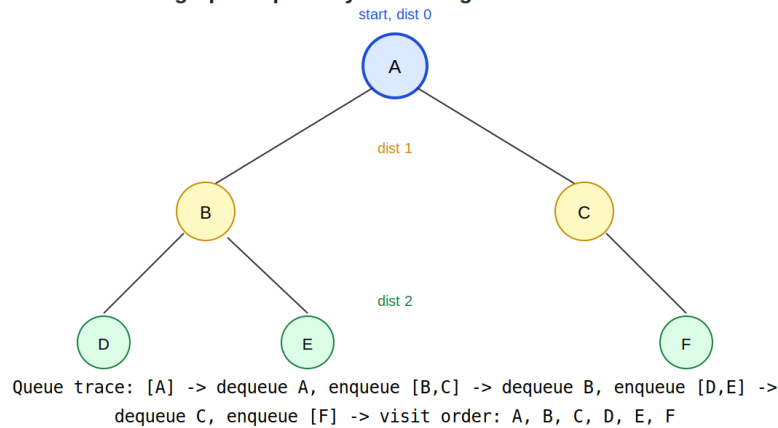
Simple model: Start by enqueueing the starting node and marking it visited. Repeatedly dequeue a node, process it, and enqueue any of its unvisited neighbors (marking each as visited at the moment it's enqueued, not when it's later dequeued). This produces the same level-by-level guarantee as 8.3's tree version, extended to handle graphs with cycles and multiple paths between nodes.

Next layer — the critical “mark visited at enqueue time” detail, and why BFS guarantees shortest paths:

- **Marking a node visited when it's enqueued (not when it's later dequeued and processed) is essential** — if you waited until dequeuing to mark it visited, the same node could be enqueued multiple times by different neighbors discovering it simultaneously, before any of those enqueue operations have had a chance to mark it, wasting work and potentially breaking the algorithm's correctness on heavily-connected graphs.
- **BFS guarantees the shortest path (by edge count) in an unweighted graph** specifically because of its level-by-level exploration order: the first time a node is reached, it's guaranteed to be via the shortest possible path, since BFS exhausts every shorter-distance possibility before considering anything farther away — this is precisely why 10.2 flagged BFS as invalid for weighted graphs, where “fewest edges” and “lowest total cost” aren't the same thing.
- **Tracking the actual shortest path** (not just its length) requires storing a “parent” pointer for each node as it's discovered (which node enqueued it), then reconstructing the path backward from the target to the start once BFS completes — a common, practical extension beyond just computing distance.
- This same mechanism directly underlies “shortest path in an unweighted graph,” “minimum number of moves/steps” puzzle problems, and “is there a path at all” connectivity questions (a simplified version of 10.6's connected components, applied to just one specific pair of nodes).

5. VISUAL REPRESENTATION

BFS on a graph: explore by increasing distance from the start



Every node at distance 1 visited before ANY node at distance 2 -- guaranteed by the queue

The diagram traces BFS exploring outward from node A: nodes B and C (distance 1) are both fully processed before nodes D, E, F (distance 2) are even considered, exactly the guaranteed level-by-level order the queue mechanism produces — directly mirroring 8.3's tree-specific version, just on a graph structure with potentially more complex connectivity.

6. WHEN TO USE / WHEN NOT TO USE

Use BFS when:

- You need the shortest path (by number of edges) in an unweighted graph, or need to confirm two nodes are connected via the shortest possible route.
- You need to explore a graph level by level explicitly — “find everything reachable within k steps,” “find the minimum number of moves to reach a target state” (a very common puzzle-problem framing).

Wrong choice when:

- The graph is weighted and edge costs genuinely differ — BFS's “shortest by edge count” guarantee doesn't translate to “shortest by total cost”; Dijkstra's algorithm (Tier 2) is the correct tool there.
- You need to explore deeply along one path before backtracking (detecting all paths, certain backtracking-style enumeration problems) — that's DFS's (10.4) natural strength, not BFS's.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — basic BFS traversal on a general graph, the exact trace above, with the explicit visited set general graphs require:

```

List<int> BFS(Dictionary<int, List<int>> adjList, int start) {
    var result = new List<int>();
    var visited = new HashSet<int> { start }; // mark at ENQUEUE time
    var queue = new Queue<int>();
    queue.Enqueue(start);

    while (queue.Count > 0) {
        int node = queue.Dequeue();
        result.Add(node);
        foreach (int neighbor in adjList[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor); // mark BEFORE enqueueing
                queue.Enqueue(neighbor);
            }
        }
    }
    return result;
}

```

INTERMEDIATE — BFS for shortest path distance, tracking distance alongside each node:

```

int ShortestDistance(Dictionary<int, List<int>> adjList, int start,
                    int target) {
    var visited = new HashSet<int> { start };
    var queue = new Queue<(int node, int dist)>();
    queue.Enqueue((start, 0));

    while (queue.Count > 0) {
        var (node, dist) = queue.Dequeue();
        if (node == target) return dist;

        foreach (int neighbor in adjList[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor);
                queue.Enqueue((neighbor, dist + 1));
            }
        }
    }
    return -1; // target unreachable
}

```

ADVANCED — BFS with full path reconstruction, using parent pointers to recover the actual shortest path, not just its length:

```

List<int> ShortestPath(Dictionary<int, List<int>> adjList, int start,
                      int target) {
    var visited = new HashSet<int> { start };
    var parent = new Dictionary<int, int>();
    var queue = new Queue<int>();
    queue.Enqueue(start);

    while (queue.Count > 0) {
        int node = queue.Dequeue();
        if (node == target) break;

        foreach (int neighbor in adjList[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor);
                parent[neighbor] = node;    // record HOW we got here
                queue.Enqueue(neighbor);
            }
        }
    }

    if (!visited.Contains(target)) return new List<int>(); // unreachable

    var path = new List<int>();
    int curr = target;
    while (curr != start) {
        path.Add(curr);
        curr = parent[curr];
    }
    path.Add(start);
    path.Reverse();    // built backward, reverse for start->target order
    return path;
}

```

This combines BFS's traversal with a parent dictionary tracking exactly how each node was first reached — reconstructing the path means walking backward from the target through these parent links until reaching the start, then reversing the result, since the natural reconstruction order is target-to-start, the opposite of what's wanted.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, BFS generalizes into multi-source BFS (starting the search from multiple nodes simultaneously, all enqueued at distance 0 before the search begins) — a genuinely useful variant for problems like “find the distance from every cell to its nearest target cell” (e.g., a grid with multiple fire sources, finding when each cell catches fire), which is far more efficient than running separate single-source BFS from every target and taking the minimum. There's also a connection worth flagging to 9.2's PriorityQueue: Dijkstra's algorithm (Tier 2) can be understood as “BFS, but using a priority queue ordered by accumulated distance instead of a plain FIFO queue” — once you understand BFS's mechanism deeply, Dijkstra's is a natural, well-motivated extension rather than an unrelated new algorithm.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is marking a node as visited when it's dequeued rather than when it's enqueued — on a graph (unlike a tree, where each node has exactly one path from the root), this can cause the same node to be enqueued multiple times by different neighbors before any processing happens, wasting work and potentially producing incorrect distance calculations. Another is forgetting the explicit visited set entirely, assuming (incorrectly) that graphs are acyclic the way trees are — without this tracking, BFS on a graph with a cycle would loop forever, re-enqueueing already-explored nodes indefinitely. A third: applying BFS to a weighted graph and treating the resulting “shortest path by edge count” as if it were “shortest path by total cost”

— a subtle, easy-to-miss error that produces a plausible-looking but genuinely wrong answer whenever edge weights aren't uniform.

10. SELF-CHECK

- Why must a node be marked visited at the moment it's enqueued, rather than waiting until it's later dequeued and processed?
- Walk through, out loud, why BFS guarantees that the first time a node is reached, it's via the shortest possible path (by edge count) — what specific property of the level-by-level exploration order guarantees this?
- In the path-reconstruction example, why does walking backward from the target through parent pointers produce the path in target-to-start order, requiring an explicit reversal at the end?

10.4 — Depth-First Search (DFS) — mechanics and use cases (recursive and iterative)

1. PLAIN-LANGUAGE GIST

DFS explores a graph by plunging as deep as possible down one path before backtracking to try alternatives — the opposite exploration order from BFS's level-by-level expansion — using either natural recursion (the call stack implicitly tracking the path) or an explicit stack (4.1's pattern), and like BFS, requires an explicit “visited” set on general graphs to avoid infinite loops on cycles.

2. REAL-WORLD ANALOGY

Picture exploring a maze by always picking a direction and walking as far as that path allows, only turning back (backtracking) once you hit a dead end or a place you've already been — rather than methodically checking every nearby room before going further in any direction (which would be BFS's strategy instead). DFS commits fully to one direction before ever reconsidering alternatives.

3. CONNECT TO PRIOR KNOWLEDGE

This is the direct generalization of 8.2's tree-specific depth-first traversals (preorder/inorder/postorder) and 5.2's backtracking “try, recurse, undo” pattern — DFS on a general graph is structurally identical to those tree traversals and backtracking explorations, just requiring 2.5's HashSet-based visited-tracking (which trees never needed, since trees have no cycles) to stay correct on graphs that can revisit nodes via different paths.

4. CORE MECHANISM (Simple → Intermediate)

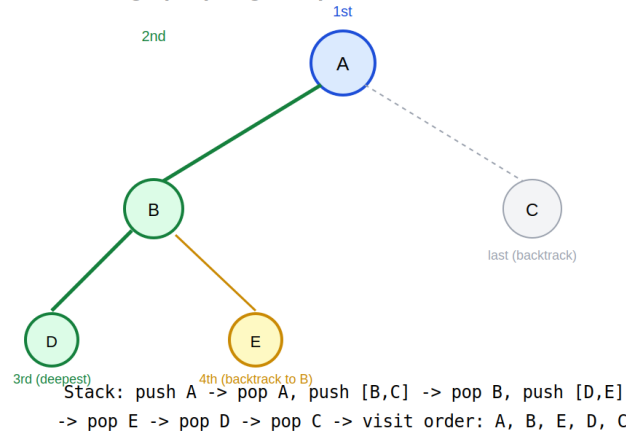
Simple model: Starting from a node, mark it visited, then recursively visit each of its unvisited neighbors, one at a time, fully exploring each neighbor's entire reachable subgraph before moving to the next neighbor. The recursive call stack naturally tracks “the path taken to get here,” with backtracking happening automatically as recursive calls return.

Next layer — recursive vs. iterative implementation, and the genuinely new requirement graphs introduce:

- **Recursive DFS** directly mirrors 8.2's tree traversal code shape — the function calls itself on each unvisited neighbor, and the implicit call stack handles all the “remember where to backtrack to” bookkeeping automatically, exactly as it did for trees.
- **Iterative DFS** uses an explicit `Stack<T>` (4.1's pattern) instead of relying on the recursive call stack — useful when recursion depth risks a stack overflow on a very large or deeply connected graph, or when an interviewer specifically wants to see the underlying mechanism made explicit rather than delegated to the language's call stack.
- **The critical new requirement, compared to tree traversal:** a general graph can have cycles and multiple paths to the same node, so DFS must track visited nodes explicitly (2.5's HashSet pattern) — without this, DFS on a graph with a cycle would recurse forever, endlessly re-exploring the same loop. Trees never needed this because a tree's acyclic structure guarantees you can never revisit a node by construction.
- **Marking visited before recursing into a node** (not after) is the safe, standard convention — marking too late can allow a node to be entered multiple times simultaneously through different active recursive paths before any of them get a chance to record the visit, the graph-DFS analogue of BFS's “mark at enqueue time, not dequeue time” rule from 10.3.

5. VISUAL REPRESENTATION

DFS on the same graph: plunge deep down ONE branch before backtracking



DFS fully exhausts one branch (D, E under B) before ever reaching C -- the opposite of BFS

The diagram traces DFS on the same graph structure used in 10.3's BFS example: starting at A, DFS immediately commits to exploring B's entire branch (down to D and E) before ever backtracking to consider C — a visibly different exploration order from BFS's level-by-level pattern on the identical graph.

6. WHEN TO USE / WHEN NOT TO USE

Use DFS when:

- You need to explore all paths or all reachable nodes without caring about finding the shortest path specifically — DFS doesn't have BFS's shortest-path guarantee, but it's often simpler and uses less memory ($O(\text{depth})$ for the stack, versus BFS's potentially $O(\text{width})$ queue) when shortest-path isn't the goal.
- You're solving a problem with a natural “explore fully, then backtrack” structure — detecting cycles (10.5), finding connected components (10.6), topological sort (10.7), or any backtracking-flavored search (5.2's pattern, now applied to graph structures specifically).

Wrong choice when:

- You specifically need the shortest path in an unweighted graph — DFS provides no such guarantee; a path DFS happens to find first could be far longer than the actual shortest one, since DFS's exploration order has nothing to do with distance from the start.
- The graph is extremely deep (a very long chain of connections) and you're using recursive DFS — this risks a real stack overflow (0.7's warning, directly applicable here), making iterative DFS with an explicit stack the safer choice for such inputs.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — recursive DFS, the exact trace above, directly mirroring 8.2's tree traversal shape:


```

void DFS(Dictionary<int, List<int>> adjList, int node,
        HashSet<int> visited, List<int> result) {
    visited.Add(node);           // mark BEFORE recursing
    result.Add(node);

    foreach (int neighbor in adjList[node]) {
        if (!visited.Contains(neighbor)) {
            DFS(adjList, neighbor, visited, result);
        }
    }
}

```

INTERMEDIATE — iterative DFS using an explicit stack, directly extending 4.1's preview to general graphs:

```

List<int> DFSIterative(Dictionary<int, List<int>> adjList, int start) {
    var result = new List<int>();
    var visited = new HashSet<int>();
    var stack = new Stack<int>();
    stack.Push(start);

    while (stack.Count > 0) {
        int node = stack.Pop();
        if (visited.Contains(node)) continue; // handled via another path

        visited.Add(node);
        result.Add(node);

        foreach (int neighbor in adjList[node]) {
            if (!visited.Contains(neighbor)) {
                stack.Push(neighbor);
            }
        }
    }
    return result;
}

```

Notice the visited-check happens after popping here (a slightly different convention than the recursive version's “mark before recursing”) — this is a valid alternative style specifically because a node might be pushed onto the stack multiple times via different paths before any of them are popped; the `if (visited.Contains(node)) continue` guard handles that safely by simply skipping redundant pops rather than preventing the redundant pushes in the first place.

ADVANCED — DFS used to find all paths between two nodes (not just one), directly reusing 5.2's backtracking try/recurse/undo template:

```

void FindAllPaths(Dictionary<int, List<int>> adjList, int current,
                  int target, HashSet<int> visited, List<int> path,
                  List<List<int>> allPaths) {
    visited.Add(current);           // TRY
    path.Add(current);

    if (current == target) {
        allPaths.Add(new List<int>(path)); // found one -- COPY it
    } else {
        foreach (int neighbor in adjList[current]) {
            if (!visited.Contains(neighbor)) {
                FindAllPaths(adjList, neighbor, target, visited,
                             path, allPaths); // RECURSE
            }
        }
    }

    path.RemoveAt(path.Count - 1); // UNDO
    visited.Remove(current);        // UNDO -- allows revisit via
                                    // a DIFFERENT path
}

```

The critical detail distinguishing this from simple connectivity-checking DFS:

`visited.Remove(current)` in the undo step — since you're enumerating all paths (not just checking reachability once), a node visited along one explored path must become available again for a different path that might legitimately revisit it; this is exactly 5.2's backtracking discipline applied to graph traversal, where the undo step is what makes correct full enumeration possible.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, DFS is the algorithmic foundation behind computing a graph's DFS tree/forest, discovery and finish times for each node (timestamps marking when DFS first visits and finally leaves each node), which together enable classifying edges (tree edges, back edges, forward edges, cross edges) — a more advanced framework used to prove correctness of cycle detection (10.5) and topological sort (10.7) rigorously, beyond just the practical mechanics covered here. There's also a direct connection worth flagging: DFS's “fully explore, then backtrack” structure is mechanically identical to 5.2's backtracking template applied to an explicit graph rather than an abstract decision tree — recognizing graph DFS and decision-tree backtracking as the same underlying pattern, just operating on different kinds of “nodes,” is a valuable unifying insight.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming DFS finds the shortest path, the way BFS reliably does — DFS's exploration order is determined by which neighbor happens to be visited first, with no relationship to path length, so a DFS-found path between two nodes could be dramatically longer than the true shortest path. Another is forgetting to un-mark a node as visited when the goal is enumerating all paths (rather than just checking connectivity once) — without removing the node from the visited set during backtracking (the advanced example's `visited.Remove(current)`), legitimate alternative paths that should revisit that node get incorrectly blocked. A third: using recursive DFS on an extremely deep or large graph without considering stack overflow risk — unlike trees (which are often shallow, $O(\log n)$, if balanced per 8.1), a graph's DFS recursion depth can be as large as the total number of nodes in a worst-case “long chain” structure, making the iterative, explicit-stack version the safer default for graphs of unknown or potentially large depth.

10. SELF-CHECK

- Why does DFS not guarantee finding the shortest path between two nodes, even though it's guaranteed to find a path if one exists?
- In the “find all paths” example, why is `visited.Remove(current)` necessary during backtracking, when the basic connectivity-checking version of DFS doesn't need this undo step?
- Why might iterative DFS (using an explicit stack) be preferred over recursive DFS specifically for very large or deeply connected graphs?

10.5 — Detecting cycles in a graph

1. PLAIN-LANGUAGE GIST

Detecting whether a graph contains a cycle — a path that loops back to a node already on the current path — requires genuinely different logic depending on whether the graph is directed or undirected, because an undirected graph's edges naturally let DFS “revisit” the node it just came from (which is NOT a cycle, just walking back along the same edge), while a directed graph has no such automatic revisit, making any repeated node along the current path a real cycle.

2. REAL-WORLD ANALOGY

Picture walking along a two-way street and then walking back the way you came — you've revisited your starting point, but you haven't actually looped around anything; you just turned back. Now picture a one-way street system where you find yourself back at an intersection you already passed through while still moving forward — that genuinely means the streets loop around in a circle, a real cycle, since there's no “walking backward” possible to explain the repeat innocently.

3. CONNECT TO PRIOR KNOWLEDGE

This directly builds on 10.4's DFS mechanics, adding a specific bookkeeping requirement on top — and it explicitly resolves the structural distinction flagged in 10.2 between directed and undirected graphs, showing concretely why that classification isn't just a labeling exercise but actually changes which algorithm is correct.

4. CORE MECHANISM (Simple → Intermediate)

Simple model — directed graphs: Track two separate sets during DFS: an overall visited set (nodes fully processed at some point) and a currentPath set (nodes on the active recursive call stack right now, i.e., still being explored). If DFS encounters a neighbor that's in currentPath, that's a genuine cycle — you've looped back to something still “in progress.” If DFS encounters a neighbor that's in visited but not in currentPath, that's just a previously-explored, now-finished branch — not a cycle, simply a different valid path converging.

Simple model — undirected graphs: Track visited and, for each DFS call, the parent (the node you arrived from). If DFS encounters a neighbor that's already visited and that neighbor is not the immediate parent, that's a genuine cycle. If the visited neighbor IS the parent, that's just the edge you arrived on, traversed backward — not a cycle.

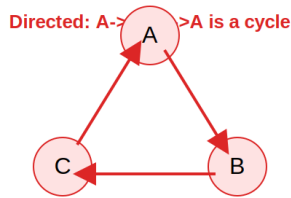
Next layer — why “just visited” isn't enough for directed graphs, and why “parent-check” suffices for undirected:

- For a DIRECTED graph, simply checking “have I seen this node before, anywhere in the whole traversal” is NOT sufficient — a directed acyclic graph (DAG) can easily have multiple paths converging on the same node (e.g., two different prerequisite chains both leading to the same downstream course) without any actual cycle; the currentPath set specifically distinguishes “this node is part of my currently active exploration chain” (a real cycle if revisited) from “this node was visited via some completely different, already-finished branch” (not a cycle at all).
- For an UNDIRECTED graph, every edge is inherently traversed “both ways” conceptually — DFS exploring from A to B will naturally see B's edge back to A, which looks like a revisit but isn't a meaningful cycle, just the same single edge viewed from both ends; tracking the immediate parent specifically filters out this single, harmless case, while still catching genuine cycles (any revisit to an already-visited, non-parent node).

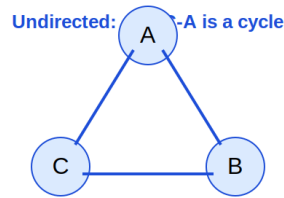
- This distinction is precisely why 10.2 emphasized that directed/undirected isn't a superficial label — it fundamentally changes which bookkeeping (a currentPath set vs. a parent reference) is needed for this exact problem.

5. VISUAL REPRESENTATION

Cycle detection needs different logic for directed vs undirected graphs



Needs tracking "currently in recursion stack" (a node revisited WHILE still being explored = cycle). Just "visited" isn't enough -- DAGs revisit too.



Just track "visited" AND "parent" -- revisiting a visited node that ISN'T your immediate parent means a cycle (since edges are bidirectional).

Undirected DFS naturally "revisits" the parent via the same edge traversed backward -- that's NOT a cycle, just the edge you came from. Directed graphs don't have this issue since edges only go one way.
This is why the two cases need genuinely different detection algorithms.

The diagram shows a directed cycle ($A \rightarrow B \rightarrow C \rightarrow A$) requiring currentPath tracking to correctly identify, alongside an undirected cycle ($A-B-C-A$) requiring only a parent check — illustrating why the undirected case can get away with much simpler bookkeeping, since its "revisit the parent" case is the only false positive that needs filtering out.

6. WHEN TO USE / WHEN NOT TO USE

Apply directed cycle detection (currentPath tracking) when:

- You need to verify a directed graph is acyclic before applying topological sort (10.7), which is only well-defined on a DAG.
- You're checking for circular dependencies in a directed structure — build systems, course prerequisites, task scheduling with "must happen before" relationships.

Apply undirected cycle detection (parent tracking) when:

- You're checking whether an undirected graph has any redundant connections (a cycle implies there's more than one path between some pair of nodes) — relevant to certain spanning-tree algorithms (Tier 2/3) that specifically need a cycle-free (acyclic) subset of edges.

Wrong choice / traps to avoid:

- Applying the undirected "parent-check" logic to a directed graph — this would incorrectly treat legitimate converging paths as non-cycles or miss certain genuine cycles, since directed graphs don't have the "edges are symmetric" property that makes parent-checking sufficient.
- Applying the directed "currentPath" logic to an undirected graph without modification — while not necessarily wrong, it's unnecessarily complex bookkeeping for a problem that the simpler parent-check approach handles directly.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — undirected cycle detection via parent-tracking, the exact mechanism described above:

```

bool HasCycleUndirected(Dictionary<int, List<int>> adjList, int node,
                        int parent, HashSet<int> visited) {
    visited.Add(node);
    foreach (int neighbor in adjList[node]) {
        if (!visited.Contains(neighbor)) {
            if (HasCycleUndirected(adjList, neighbor, node, visited))
                return true;
        } else if (neighbor != parent) {
            return true;    // visited, and NOT the parent -- real cycle
        }
    }
    return false;
}

```

INTERMEDIATE — directed cycle detection via the currentPath set, the exact mechanism described above:

```

bool HasCycleDirected(Dictionary<int, List<int>> adjList, int node,
                      HashSet<int> visited, HashSet<int> currentPath) {
    visited.Add(node);
    currentPath.Add(node);    // mark as "actively being explored"

    foreach (int neighbor in adjList[node]) {
        if (currentPath.Contains(neighbor)) {
            return true;    // neighbor is ACTIVE path -- real cycle
        }
        if (!visited.Contains(neighbor)) {
            if (HasCycleDirected(adjList, neighbor, visited, currentPath))
                return true;
        }
        // visited but NOT in currentPath: finished branch -- fine
    }

    currentPath.Remove(node);    // done exploring -- UNDO
    return false;
}

```

The `currentPath.Remove(node)` at the end is essential — exactly 5.2's backtracking discipline again — without removing the node from `currentPath` once its exploration is fully finished, later, entirely unrelated branches would incorrectly see it as “still active” and report false cycles.

ADVANCED — detecting a cycle in a directed graph using Kahn's algorithm's in-degree approach instead of DFS:

```

bool HasCycleDirectedKahn(int numNodes, Dictionary<int, List<int>> adjList) {
    int[] inDegree = new int[numNodes];
    foreach (var neighbors in adjList.Values) {
        foreach (int neighbor in neighbors) inDegree[neighbor]++;
    }

    var queue = new Queue<int>();
    for (int i = 0; i < numNodes; i++) {
        if (inDegree[i] == 0) queue.Enqueue(i); // no prerequisites
    }

    int processedCount = 0;
    while (queue.Count > 0) {
        int node = queue.Dequeue();
        processedCount++;
        foreach (int neighbor in adjList[node]) {
            if (--inDegree[neighbor] == 0) queue.Enqueue(neighbor);
        }
    }

    return processedCount != numNodes; // stuck nodes -- a cycle exists
}

```

This is a genuinely different strategy from DFS-based detection: if every node can eventually be processed (reach in-degree zero and get dequeued), the graph is acyclic; if some nodes are permanently stuck with positive in-degree (because they're part of a cycle, perpetually waiting on each other), processedCount falls short of numNodes, revealing a cycle without ever explicitly tracking a “current path” — this is precisely the mechanism 10.7's topological sort builds on directly.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, the directed cycle detection technique shown here (tracking visited and currentPath together) is formally connected to the edge classification framework mentioned in 10.4's deeper knowledge — a “back edge” (one pointing to an ancestor still on the current DFS path) is precisely what indicates a cycle, and this terminology is the standard vocabulary used when discussing cycle detection rigorously in algorithms literature. There's also a practical generalization worth knowing: detecting cycles in a directed graph is closely related to detecting deadlocks in resource-allocation systems (where processes waiting on each other's resources form a directed graph, and a cycle in that graph indicates an unresolvable deadlock) — the exact same cycle-detection algorithm shown here is a real, applied technique in operating systems and concurrent programming contexts.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is using a single visited set for directed cycle detection without the separate currentPath tracking — this conflates “previously explored via some other branch” with “currently on my active path,” causing false-positive cycle reports on perfectly valid DAGs where multiple paths legitimately converge on the same node. Another is forgetting to remove a node from currentPath once its exploration is complete (the backtracking “undo” step) — without this, the currentPath set incorrectly grows to include nodes from already-finished branches, causing false cycle detections in later, unrelated parts of the traversal. A third: applying the undirected graph's simpler parent-check logic to a directed graph, assuming the two cycle-detection problems are interchangeable — they require fundamentally different bookkeeping precisely because directed and undirected graphs have structurally different notions of what counts as a “harmless revisit” versus a genuine cycle.

10. SELF-CHECK

- Why is checking only a visited set (without currentPath) insufficient for detecting cycles in a directed graph, even though it works for simpler reachability checks?
- Walk through, out loud, why an undirected graph's cycle detection can get away with just tracking the immediate parent, while a directed graph needs to track the entire currently-active path.
- In Kahn's algorithm-based cycle detection, why does `processedCount != numNodes` at the end specifically indicate the presence of a cycle, without ever explicitly tracking which nodes are “currently being explored”?

10.6 — Connected components

1. PLAIN-LANGUAGE GIST

A connected component is a maximal group of nodes where every node can reach every other node within that same group via some path — and a single graph variable can secretly contain multiple, entirely separate components (“islands”) with no edges connecting them at all, which is detected by repeatedly running BFS or DFS from any not-yet-visited node and counting how many separate explorations it takes to cover the entire graph.

2. REAL-WORLD ANALOGY

Picture a map showing several islands, each with its own internal network of roads connecting every location on that island to every other location on the same island — but with no bridges between islands at all. Each island is one connected component: thoroughly internally connected, but completely isolated from every other island. Counting “how many separate landmasses are there” is exactly the connected-components question.

3. CONNECT TO PRIOR KNOWLEDGE

This is a direct, practical application of either 10.3's BFS or 10.4's DFS — the core mechanism is identical to ordinary graph traversal, with the only new idea being to run that traversal repeatedly, once per not-yet-discovered “island,” rather than assuming a single traversal from one arbitrary starting point will reach the entire graph (which it only will if the graph happens to be a single connected component).

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Loop over every node in the graph. If a node hasn't been visited yet, it must belong to a component nobody has discovered so far — run a fresh BFS or DFS starting from it, marking everything reachable as visited (and as belonging to this new component), and increment a component counter. Continue the outer loop; any node already marked visited is skipped, since it's already known to belong to some previously-discovered component.

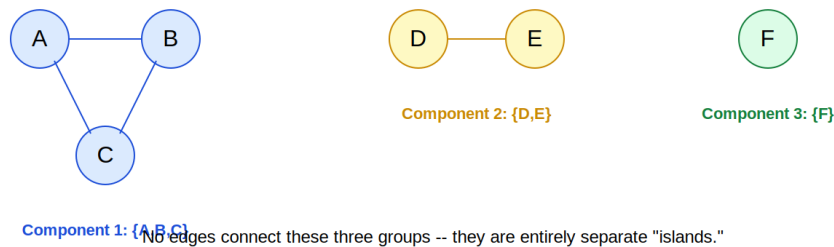
Next layer — why this correctly finds every component, and what “maximal” actually means:

- Any single BFS/DFS run starting from one node only ever explores that node's own component — it's structurally impossible for that traversal to “leak” into a different component, since by definition there are no edges connecting separate components for the traversal to follow.
- The outer loop over all nodes is what guarantees every component gets found — without it, you'd only ever discover whichever single component happened to contain your one arbitrary starting node, completely missing any other “islands” that might exist in the same graph structure.
- “Maximal” means the component includes absolutely everything reachable, not just some subset — a single BFS/DFS run naturally produces this, since it only stops expanding when there's truly nothing more reachable left to explore, not based on some arbitrary cutoff.
- This generalizes directly to “are these two specific nodes connected” (a simpler, more targeted question than counting all components) — just run BFS/DFS from one of the two nodes and check whether the other ends up in the visited set, without needing the full outer loop over every node in the graph.

5. VISUAL REPRESENTATION

Connected components: separate "islands" within one graph structure

A single graph variable can secretly contain multiple disconnected pieces:



Algorithm: loop over EVERY node; if not yet visited, run a fresh BFS/DFS from it (marking everything reachable as one component), and increment a component counter. Repeat until every node has been visited once.

A single BFS/DFS from one start node only ever explores ITS OWN component.

The diagram shows three separate connected components within a single graph structure: {A, B, C}, {D, E}, and {F} — no edges connect these groups to each other, meaning three independent BFS/DFS runs (one started from a node in each group) are needed to discover the graph's complete structure.

6. WHEN TO USE / WHEN NOT TO USE

Use connected-components analysis when:

- You need to count or identify separate, disconnected groups within a graph — social network "friend circles" that don't overlap, identifying separate clusters in a network, checking if a graph is "fully connected" (exactly one component) or fractured into multiple pieces.
- You need to answer "are these two nodes connected at all" as a simpler, special case — running a single traversal from one node and checking membership of the other in the visited set, without needing the full component-counting loop.

Wrong choice when:

- You only ever expect (or have already verified) that the graph is a single connected component, and you genuinely don't need to handle the multiple-components case — in that scenario, a single BFS/DFS run from any starting node suffices, and the extra outer-loop bookkeeping is unnecessary overhead.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — counting the number of connected components, the exact scenario diagrammed above:

```

int CountConnectedComponents(int numNodes, Dictionary<int, List<int>> adjList) {
    var visited = new HashSet<int>();
    int componentCount = 0;

    for (int node = 0; node < numNodes; node++) {
        if (!visited.Contains(node)) {
            componentCount++;
            BFS(adjList, node, visited); // mark this whole component
        }
    }
    return componentCount;
}

void BFS(Dictionary<int, List<int>> adjList, int start, HashSet<int> visited) {
    var queue = new Queue<int>();
    queue.Enqueue(start);
    visited.Add(start);

    while (queue.Count > 0) {
        int node = queue.Dequeue();
        foreach (int neighbor in adjList[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor);
                queue.Enqueue(neighbor);
            }
        }
    }
}

```

INTERMEDIATE — checking if two specific nodes are connected, the simpler targeted variant:

```

bool AreConnected(Dictionary<int, List<int>> adjList, int nodeA, int nodeB) {
    var visited = new HashSet<int> { nodeA };
    var queue = new Queue<int>();
    queue.Enqueue(nodeA);

    while (queue.Count > 0) {
        int node = queue.Dequeue();
        if (node == nodeB) return true;

        foreach (int neighbor in adjList[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor);
                queue.Enqueue(neighbor);
            }
        }
    }
    return false; // exhausted nodeA's component without finding nodeB
}

```

ADVANCED — labeling every node with its component ID, a common, practically useful extension beyond just counting:

```

Dictionary<int, int> LabelComponents(int numNodes,
    Dictionary<int, List<int>> adjList) {
    var componentId = new Dictionary<int, int>();
    int currentId = 0;

    for (int node = 0; node < numNodes; node++) {
        if (!componentId.ContainsKey(node)) {
            var queue = new Queue<int>();
            queue.Enqueue(node);
            componentId[node] = currentId;

            while (queue.Count > 0) {
                int curr = queue.Dequeue();
                foreach (int neighbor in adjList[curr]) {
                    if (!componentId.ContainsKey(neighbor)) {
                        componentId[neighbor] = currentId;
                        queue.Enqueue(neighbor);
                    }
                }
            }
            currentId++;
        }
    }
    return componentId;
}

```

This replaces the plain visited HashSet with a Dictionary<int,int> mapping each node directly to which component it belongs to — a small structural change that turns “just count the components” into “tell me exactly which component every single node is part of,” a genuinely more useful result for many real applications.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, connected components in an undirected graph have a more refined, more complex counterpart in directed graphs called strongly connected components — groups of nodes where every node can reach every other node and vice versa, respecting edge direction both ways — requiring more sophisticated algorithms (Tarjan's or Kosaraju's algorithm, Tier 3 topics) than the simple BFS/DFS approach sufficient for undirected components. There's also a practical connection worth knowing: the Union-Find / Disjoint Set Union structure (previewed by name in 2.5's deeper knowledge) is an alternative, often faster way to track and merge connected components incrementally as edges are added one at a time, without needing to re-run a full BFS/DFS traversal from scratch after every single edge addition — particularly valuable when edges arrive dynamically over time rather than all being known upfront.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is running only a single BFS/DFS from one arbitrary starting node and assuming it covers the entire graph — this silently misses any other components that might exist, producing a structurally incomplete (and often badly wrong) result whenever the graph isn't a single connected piece, which is a genuinely common real-world scenario, not a rare edge case. Another is forgetting the outer loop over all nodes, or implementing it incorrectly (e.g., looping only over nodes with at least one edge, missing isolated single-node components like node “F” in the diagram, which has no edges at all but still counts as its own component). A third: using a generic visited set when the actual problem needs to know which component each node belongs to (not just whether it's been visited) — missing the easy upgrade to a component-labeling Dictionary (the advanced example) when the problem's real requirement calls for that richer information.

10. SELF-CHECK

- Why does a single BFS or DFS run, starting from one node, only ever explore that node's own connected component, never “leaking” into a different one?
- Why is the outer loop over every node in the graph essential for correctly counting all connected components, rather than just running one traversal and calling it done?
- In the component-labeling example, why does switching from a plain `HashSet<int>` (just tracking visited) to a `Dictionary<int,int>` (tracking which component each node belongs to) provide meaningfully more useful information for many real applications?

10.7 — Topological sort (concept and use cases)

1. PLAIN-LANGUAGE GIST

A topological sort produces a valid linear ordering of a directed acyclic graph's nodes such that every edge points “forward” in that ordering — if there's an edge from A to B, A must come before B in the resulting sequence — which is exactly the right tool for any “must happen before” dependency problem, like course prerequisites or build-system task ordering.

2. REAL-WORLD ANALOGY

Picture a list of college courses with prerequisites: Calculus I must come before Calculus II, Calculus I must come before Physics I, and so on. A topological sort produces some valid order in which you could actually take all these courses without ever violating a prerequisite — and critically, there can be multiple equally valid orderings (you might take Physics I before Calculus II, or vice versa, as long as each course's actual prerequisites are satisfied before it).

3. CONNECT TO PRIOR KNOWLEDGE

This directly builds on 10.5's cycle detection (specifically the Kahn's algorithm variant previewed there) and 10.2's identification of DAGs as a specially important graph structure — a topological sort is only well-defined on a directed acyclic graph, since a cycle would create an impossible-to-satisfy circular dependency (A must come before B, but B must also come before A), which is exactly why 10.5's cycle-checking logic is a natural prerequisite skill for this topic.

4. CORE MECHANISM (Simple → Intermediate)

Simple model — Kahn's algorithm (BFS-based): Compute each node's in-degree (the number of edges pointing into it, i.e., how many prerequisites it has). Repeatedly find a node with in-degree zero (no remaining unsatisfied prerequisites), add it to the output order, then “remove” it from the graph by decrementing the in-degree of all its neighbors (since one of their prerequisites — this node — has now been satisfied). Repeat until every node has been processed.

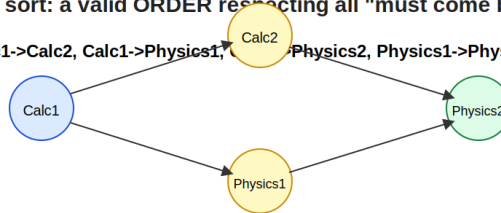
Simple model — DFS-based alternative: Run DFS from every unvisited node. Add each node to the output order after fully exploring all of its dependencies (a postorder-style placement, directly reusing 8.2's postorder concept) — then reverse the entire resulting order at the end, since postorder naturally places “dependency-heavy” nodes earlier than the things that depend on them, which is the opposite of the order actually wanted.

Next layer — why both approaches work, and the critical cycle-detection connection:

- Kahn's algorithm directly reuses the exact in-degree-tracking mechanism previewed in 10.5's advanced example — if, at the end, not every node could be processed (some remain stuck with positive in-degree because they're part of a cycle, perpetually waiting on each other), no valid topological order exists at all, and the algorithm's “stuck node” detection IS simultaneously a cycle-detection mechanism.
- The DFS-based approach's “reverse at the end” step is the part most likely to be forgotten or implemented backward — since DFS naturally finishes (and thus records) a node only after all its dependencies are already recorded, the raw output order has dependencies coming after the things that depend on them, the opposite of what's wanted; reversing restores the correct “dependencies first” order.
- Multiple valid topological orderings often exist for the same DAG (as the diagram's course-prerequisite example shows) — there's no single “correct” answer unless the DAG happens to be a single, fully linear chain with no branching; any output respecting every edge's “comes before” constraint is equally valid.

5. VISUAL REPRESENTATION

Topological sort: a valid ORDER respecting all "must come before" edges
Course prerequisites: Calc1->Calc2, Calc1->Physics1, Calc2->Physics2, Physics1->Physics2



Valid orders: Calc1, Calc2, Physics1, Physics2 OR Calc1, Physics1, Calc2, Physics2
(multiple valid orderings can exist -- topological sort isn't always unique)

Kahn's algorithm: repeatedly remove a node with in-degree 0 (no remaining prerequisites), add it to the order, then decrement its neighbors' in-degree.
DFS-based alternative: run DFS, add each node to the order AFTER all its dependencies (postorder, 8.2's pattern), then reverse the result.
Only possible on a DAG -- a cycle means no valid order can ever exist.

The diagram shows a small course-prerequisite DAG (Calc1 leading to both Calc2 and Physics1, which both lead to Physics2) — illustrating that multiple valid topological orderings exist (Calc2 and Physics1 can be taken in either relative order, as long as each comes after Calc1 and before Physics2), and that the underlying mechanism (Kahn's in-degree-zero removal, or DFS-postorder-then-reverse) both correctly handle this branching structure.

6. WHEN TO USE / WHEN NOT TO USE

Use topological sort when:

- The problem explicitly involves dependency ordering — course prerequisites, build system task ordering (“compile this file before linking”), job scheduling with “must complete before” constraints.
- You need to detect whether a set of dependencies is even satisfiable at all (no circular dependencies) — the algorithm's failure mode (not all nodes processed) doubles as a direct cycle-detection check, building on 10.5.

Wrong choice when:

- The graph isn't actually a DAG — if cycles are possible (or expected) in the input, you need to detect and handle that case explicitly (10.5) rather than assuming topological sort will simply “work”; running it on a graph with a cycle produces an incomplete, invalid result rather than a clear error unless you explicitly check for that failure mode.
- The relationships in the problem aren't genuinely about ordering/precedence at all — forcing a topological-sort framing onto a problem that's actually about something else (pure connectivity, shortest paths) misapplies a tool suited to a specific kind of dependency-ordering question.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — Kahn's algorithm, the exact mechanism described above, directly extending 10.5's cycle-detection code into a full ordering:

```

List<int> TopologicalSortKahn(int numNodes,
    Dictionary<int, List<int>> adjList) {
    int[] inDegree = new int[numNodes];
    foreach (var neighbors in adjList.Values)
        foreach (int neighbor in neighbors) inDegree[neighbor]++;

    var queue = new Queue<int>();
    for (int i = 0; i < numNodes; i++) {
        if (inDegree[i] == 0) queue.Enqueue(i);
    }

    var order = new List<int>();
    while (queue.Count > 0) {
        int node = queue.Dequeue();
        order.Add(node);
        foreach (int neighbor in adjList[node]) {
            if (--inDegree[neighbor] == 0) queue.Enqueue(neighbor);
        }
    }

    return order.Count == numNodes ? order : new List<int>(); // cycle
}

```

INTERMEDIATE — DFS-based topological sort, using postorder placement and a final reversal:

```

List<int> TopologicalSortDFS(int numNodes,
    Dictionary<int, List<int>> adjList) {
    var visited = new HashSet<int>();
    var order = new List<int>();

    for (int node = 0; node < numNodes; node++) {
        if (!visited.Contains(node)) {
            DFSHelper(adjList, node, visited, order);
        }
    }

    order.Reverse(); // flip postorder into correct dependency order
    return order;
}

void DFSHelper(Dictionary<int, List<int>> adjList, int node,
    HashSet<int> visited, List<int> order) {
    visited.Add(node);
    foreach (int neighbor in adjList[node]) {
        if (!visited.Contains(neighbor)) {
            DFSHelper(adjList, neighbor, visited, order);
        }
    }
    order.Add(node); // postorder: AFTER all neighbors are explored
}

```

ADVANCED — combining topological sort with cycle detection explicitly, producing a clear error rather than a silently incomplete result:


```

(bool isValid, List<int> order) ScheduleCourses(int numCourses,
    List<(int course, int prerequisite)> prerequisites) {
    var adjList = new Dictionary<int, List<int>>();
    for (int i = 0; i < numCourses; i++) adjList[i] = new List<int>();
    foreach (var (course, prereq) in prerequisites) {
        adjList[prereq].Add(course);    // prereq must come BEFORE course
    }

    var order = TopologicalSortKahn(numCourses, adjList);

    if (order.Count != numCourses) {
        return (false, new List<int>());    // explicit failure -- cycle
    }
    return (true, order);
}

```

This wraps the Kahn's-algorithm implementation with an explicit, named success/failure result rather than just returning an ambiguous empty list — making the cycle-detection failure mode a clear, intentional part of the function's contract, which is good practice whenever a graph algorithm's "it didn't work" case needs to be distinguished clearly from "it worked, but the answer happens to be empty."

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, topological sort is the foundational building block behind real build systems (like Make, or modern dependency-resolving package managers) — determining the correct order to compile files or install packages based on their declared dependencies is, quite literally, a topological sort problem at scale. There's also a useful refinement worth knowing: when multiple valid topological orderings exist, sometimes a problem wants a specific one according to additional criteria (e.g., lexicographically smallest) — achieved by using a priority queue (9.2) instead of a plain queue in Kahn's algorithm, always picking the smallest available in-degree-zero node first rather than an arbitrary one, directly combining this topic's mechanism with 9.2's structure.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is forgetting to check whether the algorithm actually processed every node — if `order.Count` doesn't equal the total number of nodes, the graph contains a cycle and the partial order produced is invalid and shouldn't be used as if it were a complete, correct answer. Another is forgetting the final reversal step in the DFS-based approach — submitting the raw postorder sequence (dependencies appearing after what depends on them) as if it were already the correct topological order, producing an exactly backward, invalid result. A third: assuming a topological sort is unique — many DAGs have multiple equally valid orderings, and code that expects (or tests against) one specific ordering without accounting for this can incorrectly reject a different, equally correct ordering produced by a different but equally valid implementation or traversal order.

10. SELF-CHECK

- Why is topological sort only well-defined on a directed acyclic graph, and what specifically would go wrong if you tried to apply it to a graph containing a cycle?
- Walk through, out loud, why the DFS-based topological sort approach requires reversing the postorder traversal result, rather than using that raw order directly.
- Why does Kahn's algorithm's "not all nodes were processed" failure condition double as a cycle-detection mechanism, without needing 10.5's separate `currentPath`-tracking technique?